



0107 Computer Vision

전처리

개 요

- 정의 : 기계가 이미지/영상을 이해하고 분석하는 AI기술분야
- 응용분야 : 자율 주행, 의료 영상 정밀 진단, 지능형 보안 시스템 등
- 머신러닝 파이프라인
 - 전처리 (Preprocessing): 원시 데이터를 모델이 학습할 수 있게 변환
 - 학습 (Learning): 모델이 데이터를 보고 가중치를 최적화
 - 성능 평가 (Evaluation): 학습된 모델 성능 측정
 - 추론 (Inference): 학습된 모델이 실제 입력에 대해 결과 생성
 - 후처리 (Postprocessing): 모델 출력 결과를 사용자나 서비스가 쓸 수 있게 다듬음

기하학적 변환

- 이미지의 좌표계를 조작하여 형태, 크기, 위치를 변경하는 기술
- 딥러닝 모델은 고정된 크기의 입력 텐서(Input Tensor)를 요구하는 경우가 많음
- 다양한 원본 이미지를 모델의 규격에 맞추는 작업이 선행

[핵심기법]

- 크기 조정 (Resizing) 및 다운샘플링
- 비율 유지 패딩 (Letterbox Padding)
- 이미지 피라미드 (Image Pyramid)

크기 조정 (Resizing) 및 다운샘플링

- 대부분의 CNN 모델은 구조상 고정된 입력 크기(224X224/256X256)의 이미지를 요구

- 다운샘플링(Downsampling)

- 고해상도 이미지를 모델 입력 크기에 맞춰 축소하는 과정

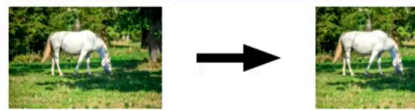
- 픽셀 정보를 압축하면서 발생하는 정보 손실을 최소화

- 앨리어싱(Aliasing) 현상을 방지하기 위해 보간법(Interpolation)이 적용

- 단순 리사이징의 한계

- 비율(Aspect Ratio)을 무시하고 강제로 정사각형으로 변환할 경우, 객체의 형태가 찌그러지는 기하학적 왜곡이 발생

- 모델이 객체의 고유한 형태적 특징을 학습하는데 심각한 방해 요인이 됨



<그림2> 이미지 리사이징
(출처: <https://medium.com/unpackai/image-preprocessing-6654d1bb4daa>)



비율 유지 패딩 (Letterbox padding)

- 왜곡문제를 해결하기 위해 '비율 유지 패딩' 기법이 널리 사용됨
- 객체 탐지(Object Detection) 모델인 YOLO(You Only Look Once) 시리즈 등에서 표준적으로 채택하는 전처리 방식
- 원본 이미지의 종횡비(Aspect Ratio)를 유지한채, 긴변을 기준으로 타겟크기에 맞춰 리사이징을 수행
- 객체의 실제 비율을 보존함으로써 모델이 객체의 형태 정보를 정확하게 학습하도록 도움
- 텍스트 인식(OCR)이나 정밀한 객체 탐지와 같이 형태의 비율이 중요한 과제에서 필수적

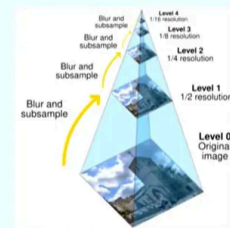


<그림 3 비율 유지 패딩 예시
(출처 : https://www.mediacollege.com/video/aspect_ratio/letterbox.html)



이미지 피라미드

- 하나의 원본 이미지를 다양한 해상도로 변환하여 계층적(Hierarchical)으로 구성하는 기법
- 단일 해상도 입력만으로는 아주 작거나 아주 큰 객체를 동시에 효과적으로 탐지하기 어려운 문제를 해결
- 다중 스케일 학습
 - 이미지 피라미드를 통해 모델은 서로 다른 해상도에서 특징(Feature)을 추출
 - 작은 객체는 고해상도 레이어에서, 큰 객체는 저해상도 레이어에서 탐지하는 (FPN(Feature Pyramid Network) 구조의 기초)
- 가우시안 블러(Gaussian Blur)와 서브샘플링을 반복하여 피라미드를 구축하는 것이 일반적



<그림 10> 이미지 피라미드 구성
(출처 : https://en.wikipedia.org/wiki/Pyramid_%28image_processing%29)

* 가우시안 블러 : 이미지 축소 시 적용하는 안티앨리어싱 필터

광도 및 색상 변환

- 이미지의 픽셀값(Intensity) 자체를 조정하여 조명 변화, 색상 왜곡, 노이즈 등 환경적 요인에 대한 모델의 강건성(Robustness)을 높이는 과정
- [핵심 기법]
- 정규화(Normalization)
- 색상 보정(Color Correction) 및 공간 변환
- 노이즈 제거(Denoising)

* 강건성(Robustness) : 입력 데이터에 예기치 못한 변화(Perturbation)나 노이즈가 발생하더라도 모델이 일관된 유지하는 능력

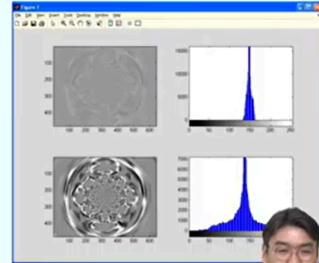


정규화

- 디지털 이미지는 통상 0에서 255 사이의 정수(Integer) 값을 가짐
- 신경망은 입력 데이터의 스케일에 매우 민감하며, 큰 값의 입력은 학습 과정에서 내부 공변량 변화(Internal Covariate Shift)를 유발하거나 최적화 과정을 불안정하게 만들 수 있음

[핵심기법]

- 스케일링(Scaling): 픽셀 값을 255로 나누어 0과 1 사이의 실수(Float) 범위로 변환, 연산의 수치적 안정성을 높임.
- 표준화(Standardization): 각 채널(R, G, B)별로 데이터의 평균을 빼고 표준편차로 나누어, 데이터 분포를 평균 0, 분산 1인 표준정규분포로 근사시킴



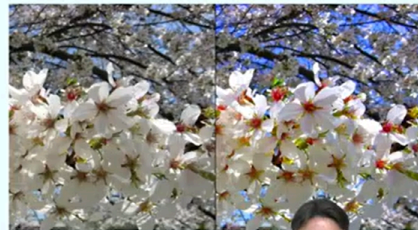
<그림4> 일반 이미지(위), 정규화된 이미지(아래)
(출처: <https://medium.com/hitchhikers-learning/9-introduction-to-deep-learning-vision-normalization-batch-normalization>)

색상 보정 및 공간 변환

- 동일한 객체라도 조명 온도(Color Temperature), 밝기, 그림자에 따라 픽셀 값(RGB)이 왜곡됨.
- 조명 변화에 흔들리지 않는 데이터 일관성(Consistency) 확보

[핵심기법]

- 히스토그램 평활화: 한쪽으로 쏠린 명암 분포를 균일하게 펼침
- 색상 공간 변환
- 밝기'와 '색상' 성분을 분리
- V(명도) 채널만 조절하여 강건한(Robust) 모델 학습 및 데이터 증강



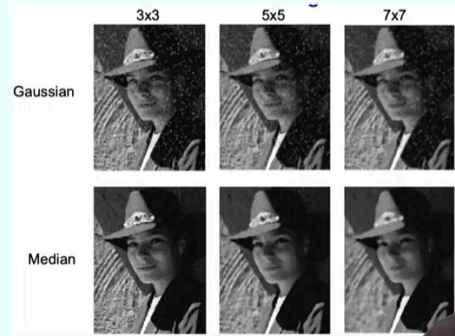
<그림5> 대비와 채도 변경을 통한 색상 보정 (출처: https://commons.wikimedia.org/wiki/File:Color_correction_in_photoshop.jpg)

노이즈 제거

- 저조도 환경(Low Light), 센서 열잡음(Thermal Noise) 등으로 불가피하게 발생
- 모델이 학습해야 할 핵심인 **고주파 성분(High Frequency: Edge, Texture)**을 훼손하여 학습 방해.

[핵심기법]

- 가우시안 필터 (Gaussian Filter)
- 중간값 필터 (Median Filter)

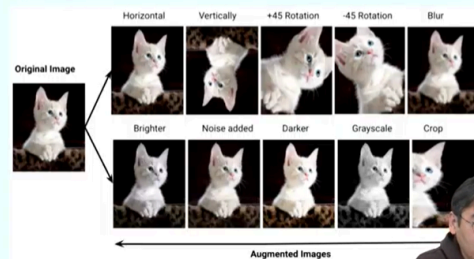


데이터 증강 전략

- 한정된 학습 데이터에 인위적인 변형을 가해 데이터의 양과 다양성을 늘리는 기법
- 훈련 데이터의 특정 패턴만 외우는 과적합을 방지
- 보지 못한 데이터에 대한 일반화(Generalization) 성능을 극대화

[핵심기법]

- 기하학적 증강
- 광도적 증강/커널 기반 증강
- 고급 증강(Advanced)
- Mosaic : 4장의 이미지를 하나로 합성
- Mixup : 두 이미지를 투명도 조절로 중첩
- CutMix : 이미지 일부를 잘라 다른 이미지로 대체



<그림 6> 데이터 증강 예시
(출처: <https://pranjal-ostwal.medium.com/data-computer-vision-b88b818b6010>)

추론(inference)

- 학습된 모델을 실전(Production)에 투입하여 새로운 데이터(Unseen Data(예측))
- 핵심 지표 : 정확도(Accuracy) + 응답 속도(Latency), 처리량(Throughput), 효율성(Efficiency)

[핵심과제]

- 이미지 분류(Image Classification)
- 이미지 분할 (Image Segmentation)
- 자세 추정(Pose Estimation)
- 객체 탐지(Object Detection)
- 얼굴 인식(Face Recognition)
- 광학문자인식(OCR)

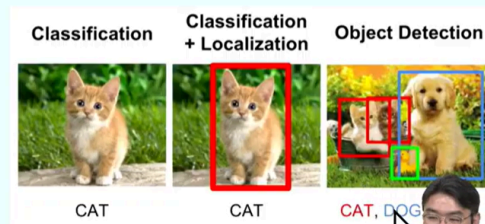
이미지 분류

이미지 분류(Image Classification)

- 입력된 이미지의 전체적인 시각정보를 분석하여 미리 정의된 정답지(Class) 중 하나를 할당하는 작업
- 화면 꼭 차게 들어온 단일 피사체를 분석할때 가장 유효

[대표적인 모델]

- ResNet(레스넷)
- EfficientNet(이피션트넷)
- ViT(비전 트랜스포머)



<그림42> 이미지 분류, 객체 탐지의 차이 (출처: <https://www.geeksforgeeks.org/what-is-image-classification/>)

객체 탐지

객체 탐지(Object Detection)

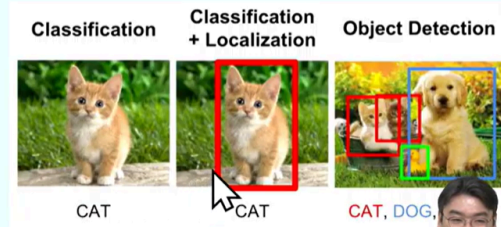
- 이미지 내에 존재하는 여러 객체의 종류(Classification)를 식별함과 동시에,
- 각 객체의 **위치(Localization)**를 바운딩 박스(Bounding Box) 형태로 찾아내는 복합적인 과제

[대표적인 모델]

- YOLO (You Only Look Once)

[활용영역]

- 자율주행, 지능형 TV, 스마트 팩토리 등



<그림 42> 이미지 분류, 객체 탐지의 차이 (출처: <https://www.geeksforgeeks.org/what-is-image-classification/>)

이미지 분할

- 이미지 내에 존재하는 여러 객체의 종류(Classification)를 식별함과 동시에,
- 각 객체의 **위치(Localization)**를 바운딩 박스(Bounding Box) 형태로 찾아내는 복합적인 과제

[기술의 목적에 따른 분류]

- 의미론적 분할(Semantic Segmentation)
- 인스턴스 분할(Instance Segmentation)

[활용영역]

- 의료영상, 자율주행의 차량 인식, 화생화학의 기상배경



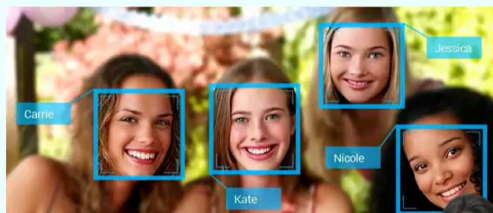
<그림 43> 이미지 분류, 객체 탐지, 이미지 분할 (출처: <https://www.ultralytics.com/ko/how-to-use-ultralytics-yolov11-for-image-classification/>)

얼굴 인식

- 이미지에서 얼굴 영역을 탐지(Detection)한 후, 눈, 코, 입 등의 랜드마크를 기준으로 정렬(Alignment)하고,
- 고유한 특징 벡터(Embedding)를 추출하여 데이터베이스와 대조함으로써 신원을 확인

[활용영역]

- 스마트폰 잠금해제
- 공항 자동 출입국 심사
- 근태관리 시스템



자세 추정

- 어깨, 팔꿈치, 무릎, 발목 등 신체의 주요 관절 포인트(Keypoints) 탐지,
- 이 점들을 연결하여 사람의 골격(Skeleton) 구조 복원
- 대상의 구체적인 행동(Action) 패턴과 자세의 정확도를 분석

[활용영역]

- AI 스포츠 코칭
- 홈트레이닝 앱
- 제스처 인터페이스



<그림 45> 자세 추정 예시 출처 :

광학 문자 인식

- 어깨, 팔꿈치, 무릎, 발목 등 신체의 주요 관절 포인트(Keypoints) 탐지,
- 이 점들을 연결하여 사람의 골격(Skeleton) 구조 복원
- 대상의 구체적인 행동(Action) 패턴과 자세의 정확도를 분석

[활용영역]

- 문서 자동화 (DX)
- 실시간 번역
- 보안 및 인증

前菜		後菜	
オリーブ2種盛り	¥580	鴨肝2種盛り	¥580
冷製毛蟹とカルパッチョ仕立て	¥1,800	小豆黒砂糖と桔梗煮	¥1,800
4種盛り	¥2,400	4種盛り	¥2,400
☆自家製黒毛和牛生ハムと		☆4種黒毛和牛生ハムと	
牛レバーのコンフィ仕立て	¥880	冷製鴨肝と鴨の肝臓と鴨の肝臓2	¥880
鶏白レバーの自家製ムース	¥850	鴨肝	¥850
ビタルス二種盛り	¥630	鴨肝と鴨の肝臓	¥630
カブリーゼサラダ	¥830	鴨肝と鴨の肝臓	¥830
パプリカとボタンのサラダ	¥760	鴨肝と鴨の肝臓	¥760
カンタリア肉アンチョビトマト	¥630	鴨肝と鴨の肝臓	¥630
☆何れも高野のサラダ	¥1,380	☆何れも高野のサラダ	¥1,380
～自家製ドレッシング～		～自家製ドレッシング～	
イタリア産チーズ2種盛り	¥1,380	イタリア産チーズ2種盛り	¥1,380
～タルトとレーズン添え～		～タルトとレーズン添え～	
☆京都府産のバーベキュー		☆京都府産のバーベキュー	
(レギュラー) 4種	¥1,180	(レギュラー) 4種	¥1,180
(ラージ) 7種	¥1,980	(ラージ) 7種	¥1,980
マッシュポテト (2人前)	¥880	マッシュポテト (2人前)	¥880
ポテトサラダ (2人前)	¥880	ポテトサラダ (2人前)	¥880

OCR 자동 번역 예시 출처 :

<https://www.sadaily.com/NewsView/990RHVZ>

후처리

- 모델이 출력한 원시적인 수치 데이터(Raw Output)를 인간이 이해할 수 있거나 시스템이 활용할 수 있는 형태로 정제하는 과정
- 서비스의 품질과 사용성을 결정짓는 마지막 관문

[핵심기법]

- 객체 탐지의 필수 후처리
 - 비최대 억제 (NMS, Non-Maximum Suppression)
 - 임계값 필터링 (Thresholding)
 - 좌표 변환 및 스케일링 (Coordinate Scaling)
 - 시각화 (Visualization)

* OCR 및 자연어 처리의 후처리



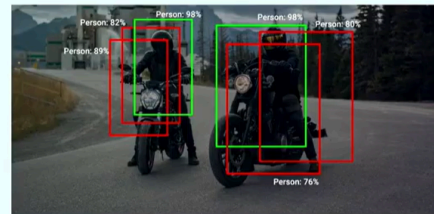
비최대 억제

- 최대값이 아닌 것들을 억제하여 중복을 제거하는 기술

[작동 원리]

- 신호도 필터링 : 점수가 특정 임계값 이하인 박스들을 제거
- 정렬 및 선정 : 남은 박스들을 점수순으로 줄 세운 뒤 / 가장 높은 점수의 박스를 '주인공'으로 선택
- IoU 기반 억제 : 인공 박스와 겹치는 영역이 많은 박스들을 / '동일 객체에 대한 중복'으로 간주하여 제거

* IoU (Intersection over Union) :



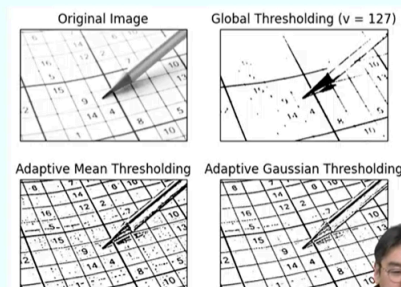
<그림 59> 비최대 억제 시각화
출처 : <https://wikidocs.net/163295>



임계값 필터링

- 불확실성의 제거 : 모델이 예측한 확률이 / 설정한 기준치 이상인 경우에만 유효한 탐지로 인정
- 결과의 정밀도(Precision) 상승 : 모델이 예측한 확률이 / 설정한 기준치 이상인 경우에만 유효한 탐지로 인정

- 좌측 상단(Original Image) : 조명이 불균일한 원본 스도쿠 사진
- 우측 상단(Global Thresholding) : 중간값 127로 필터링
- 하단 이미지(Adaptive Thresholding) : 적응형 방식



<그림 60> 임계값 필터링 예시
(출처 : <https://encord.com/blog/imagination-for-computer-vision-best-practices>)

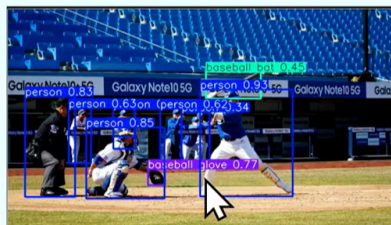


좌표 변환 및 스케일링

- '모델의 시선'과 '사용자의 시선'을 일치시키는 작업
- 학습 효율을 위해 변환된 이미지를 역변환(Invers Transform)하여 원본이미지상의 정확한 위치에 박스를 Mapping

시각화

- 데이터를 사용자가 볼 수 있는 형태로 표현하여 직관적으로 표현
- 바운딩박스 오버레이, 히트맵을 이용한 시각화 등



바운딩 박스 오버레이
(출처 : <https://www.nocutnews.co.kr/news/5314637>)



<그림 61> 히트맵을 사용한 시각화
(출처 : <https://www.labellerr.com/blog/computer-vision-based-heat-mapping-retail/>)

OCR 및 자연어 처리의 후처리 전략

- OCR 모델의 출력인 토큰 ID의 나열을 의미 있는 텍스트로 변환
- 디코딩 및 정규화
 - 토큰 디코딩: 모델의 출력 텐서를 단어 사전(Vocabulary)을 이용해 문자열로 변환(, 같은 특수 토큰 제거)
 - 텍스트 정규화: 결과의 품질을 높이기 위해 유니코드 정규화(NFC), 공백 정리, 불필요한 특수문자 제거 등을 수행
- LLM을 활용한 지능형 후처리
 - OCR이 인식한 텍스트에 오타나 문맥상 어색한 부분이 있을 경우, LLM을 통해 인식률을 비약적으로 높임

실습

01_computer_vision

1_cv_exp.ipynb

- YOLO모델을 활용한 이미지 객체 탐지
- YOLO모델을 활용한 영상 객체 탐지
- EasyOCR을 활용한 OCR 예제
- 객체 탐지 개발 파이프라인

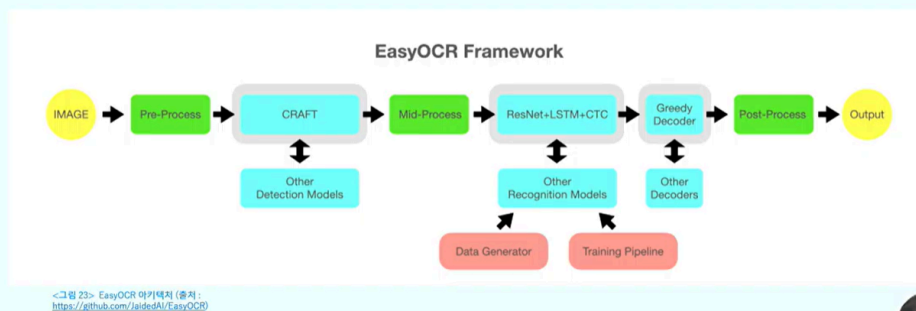
1_cv_pipeline_01_object_detection.ipynb

- YOLO를 활용한 객체 탐지 개발 파이프라인(전처리, Fine-Tuning, 성능 평가)

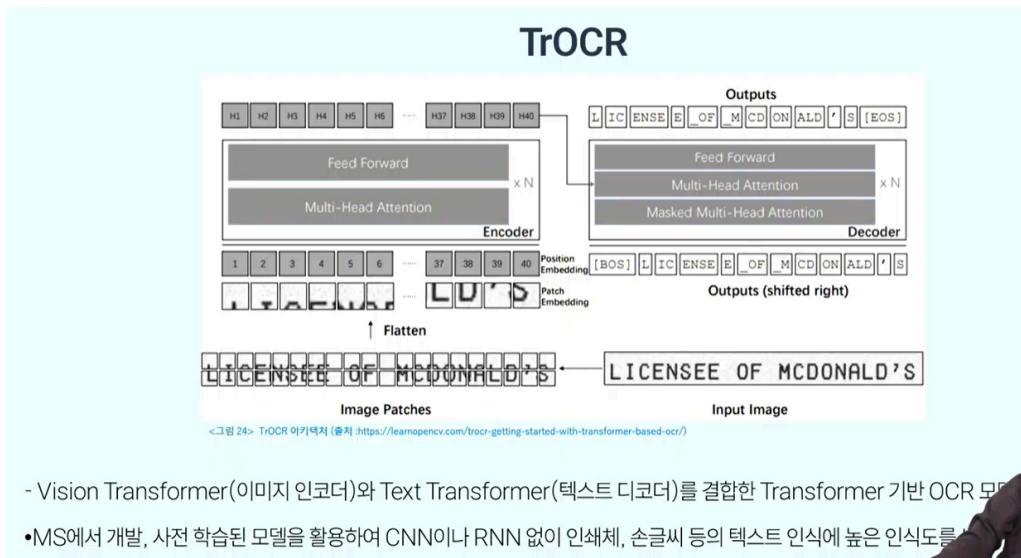
1_cv_pipeline_02_ocr.ipynb

- TrOCR을 활용한 OCR 개발 파이프라인(전처리, Fine-Tuning, 성능 평가)

EasyOCR



- CRAFT 알고리즘을 통한 텍스트 검출, CRNN(CNN+RNN+CTC) 모델을 통한 텍스트 인식 2단계로 구성
- 80개 이상 언어를 지원하고 사용이 간편한 OCR 오픈소스 라이브러리



세 개의 파일은 **컴퓨터 비전(Computer Vision)** 기술의 핵심인 ****객체 탐지(Object Detection)****와 ****광학 문자 인식(OCR)****을 다루고 있습니다.

첫 번째 파일은 이미 만들어진 모델을 ****사용(체험)****해보는 기초 실습 파일이며, 나머지 두 파일은 데이터를 가공하여 모델을 직접 ****학습(개발)****시키는 파이프라인 구축 파일입니다.

각 파일의 상세 기능은 다음과 같습니다.

▼ 1. 1_cv_exp.ipynb (기초 예제 및 실습)

이 파일은 컴퓨터 비전의 기초 기술을 빠르게 체험해볼 수 있는 **입문용 실습 노트북**입니다. 복잡한 학습 과정 없이, 사전 학습된(Pre-trained) 모델을 불러와서 이미지나 영상에 바로 적용해보는 기능을 수행합니다.

• 주요 기능:

- **이미지 객체 탐지:** YOLOv8 nano 모델을 사용하여 웹상의 이미지(예: 지단, 버스 사진)에서 사람, 넥타이, 버스 등의 객체를 찾아내고 위치를 표시합니다.
- **영상 객체 탐지:** 동영상 파일(`test_video.mp4`)을 프레임 단위로 읽어 객체를 탐지하고, 처리된 결과를 새로운 영상으로 저장합니다.
- **OCR (문자 인식):** EasyOCR 라이브러리를 활용하여 이미지 속의 한글과 영문 텍스트(예: 영수증, 간판)를 읽어내고, 텍스트의 위치와 내용을 시각화합니다.

순서	셀 종류	주요 내용 및 기능 설명
0	Markdown	노트북 소개: 객체 탐지와 OCR 기초 예제 실습임을 안내합니다.

1	Code	환경 설정: <code>computer_vision.yaml</code> 파일을 이용해 필요한 라이브러리(가상환경)를 설치합니다.
2	Code	라이브러리 импорт: OpenCV, YOLO(ultralytics), EasyOCR 등 실습에 필요한 도구들을 불러옵니다.
3	Markdown	[실습 1] 이미지 객체 탐지: YOLO 모델을 사용해 정지 이미지에서 객체를 찾는 예제 시작을 알립니다.
4	Code	이미지 객체 탐지 실행: 1. 사전 학습된 <code>yolov8n.pt</code> 모델을 로드합니다. 2. 웹상의 테스트 이미지(지단, 버스 등)를 다운로드합니다. 3. 모델에 이미지를 넣어 추론(prediction)을 수행하고 결과를 <code>runs/detect/</code> 폴더에 저장합니다.
5	Code	결과 시각화: 탐지된 결과 이미지 (<code>runs/detect/.../zidane.jpg</code>)를 화면에 띄워 눈으로 확인합니다.
6	Markdown	[실습 2] 영상 객체 탐지: 동영상 파일에서 프레임별로 객체를 탐지하는 예제입니다.
7	Code	동영상 처리 및 저장: 1. <code>test_video.mp4</code> 파일을 읽어옵니다. 2. 한 프레임씩 모델에 통과시켜 객체(사람, 자전거 등)를 탐지합니다. 3. 탐지된 박스가 그려진 프레임을 모아 새로운 영상 파일(<code>result_video.mp4</code>)로 저장합니다.
8	Markdown	[실습 3] OCR (문자 인식): 이미지 속 글자를 읽어내는 예제입니다.
9	Code	EasyOCR 실행: 1. 한국어/영어 인식을 위한 <code>EasyOCR</code> 리더를 생성합니다. 2. 영수증이나 간판 같은 이미지에서 텍스트를 읽어냅니다(<code>readtext</code>). 3. 읽어낸 글자의 위치(박스)와 텍스트 내용을 원본 이미지 위에 그려서 시각화합니다.

▼ 2. `1_cv_pipeline_01_object_detection.ipynb` (객체 탐지 개발 파이프라인)

이 파일은 단순히 모델을 사용하는 것을 넘어, **자신만의 데이터로 객체 탐지 모델을 재학습(Fine-Tuning)**시키는 전체 개발 과정을 다룹니다. YOLO 모델을 커스텀 데이터셋에 맞춰 튜닝하는 전 과정을 단계별로 수행합니다.

- **주요 기능:**

- **데이터셋 전처리:** 원본 이미지와 라벨 파일(txt)을 YOLO 학습 형식에 맞는 폴더 구조(images/train, labels/train 등)로 정리하고 분리합니다.
- **모델 학습 (Fine-Tuning):** 사전 학습된 YOLOv8 모델을 불러와서, 준비된 데이터셋으로 추가 학습을 진행합니다. 이 과정에서 `data.yaml` 설정 파일을 생성하고 활용합니다.
- **성능 평가:** 학습된 모델이 얼마나 정확한지 검증 데이터셋을 통해 평가하고, mAP(평균 정밀도) 등의 지표를 확인합니다.

순서	셀 종류	주요 내용 및 기능 설명
0	Markdown	노트북 소개: YOLO 객체 탐지 개발 파이프라인(전처리→학습→평가)을 안내합니다.
1	Markdown	[단계 1] 데이터셋 전처리: 학습용과 검증용 데이터를 나누는 단계입니다.
2	Code	데이터셋 분할 및 구조화: <code>trainval.txt</code> (학습용)와 <code>test.txt</code> (검증용) 파일 목록을 읽습니다. - 원본 이미지와 정답 라벨(txt)을 YOLO 학습 규격인 <code>images/train</code>, <code>labels/train</code> 등의 폴더 구조로 복사/이동시킵니다. - 폴더가 없으면 자동으로 생성(<code>os.makedirs</code>)합니다.
3	Markdown	[단계 2] 모델 Fine-Tuning: 데이터를 이용해 모델을 재학습시키는 단계입니다.
4	Code	YOLO 모델 학습: - 데이터셋 경로와 클래스 정보(사람, 자전거 등)가 담긴 <code>data.yaml</code> 파일을 생성합니다. <code>yolov8n.pt</code> 모델을 불러와서 설정된 데이터로 학습(<code>model.train</code>)을 시작합니다. - 학습된 최적의 모델 가중치는 <code>runs/detect/train/weights/best.pt</code> 에 저장됩니다.
5	Markdown	[단계 3] 성능 평가: 학습된 모델의 정확도를 확인하는 단계입니다.
6	Code	검증 및 지표 확인: - 학습이 완료된 <code>best.pt</code> 모델을 다시 로드합니다. - 검증 데이터셋(<code>test</code>)을 사용해 모델 성능을 측정(<code>model.val</code>)합니다. - mAP(평균 정밀도), Precision(정밀도), Recall(재

현율) 같은 수치 데이터를 출력하여 모델 성능을 평가합니다.

▼ 3. 1_cv_pipeline_02_ocr.ipynb (OCR 개발 파이프라인)

이 파일은 **OCR(문자 인식) 모델을 직접 학습**시키는 개발 파이프라인을 구축하는 파일입니다. 특히 한국어 처리에 강점이 있는 TrOCR 모델을 활용하여 AIHub 데이터셋 등으로 모델 성능을 높이는 과정을 수행합니다.

• 주요 기능:

- **TrOCR 모델 학습:** 사전 학습된 `ddobokki/ko-trocr` 모델을 기반으로, 한국어 손글씨 및 인쇄체 데이터를 학습시켜 인식률을 높입니다.
- **데이터 파이프라인 구축:** 학습용 데이터셋 클래스(`OCRDataset`)를 정의하고, 이미지와 텍스트 라벨을 모델이 학습할 수 있는 형태로 변환합니다.
- **성능 평가 및 시각화:** 학습 과정에서 CER(문자 오류율)을 계산하여 모델의 성능을 측정하고, 실제 이미지에 대한 예측 결과와 정답을 비교하여 시각화합니다.

요약하자면:

- **파일 1**은 "AI가 어떻게 동작하는지 **결과를 확인**"하는 용도이고,
- **파일 2와 3**은 "원하는 목적에 맞게 AI를 **직접 훈련**시키는 과정"을 담고 있습니다.

순서	셀 종류	주요 내용 및 기능 설명
0	Markdown	노트북 소개: TrOCR 모델을 이용한 한국어 OCR 개발 파이프라인을 안내합니다.
1	Markdown	[단계 1] TrOCR 모델 Fine-Tuning: 사전 학습 모델을 한국어 데이터로 튜닝합니다.
2	Code	라이브러리 импорт: <code>transformers</code> (Hugging Face), <code>torch</code> 등 딥러닝 라이브러리를 불러옵니다.
3	Code	데이터셋 클래스 정의 (<code>OCRDataset</code>): 1. 학습 데이터(이미지 경로, 텍스트 라벨)를 읽어오는 <code>OCRDataset</code> 클래스를 만듭니다. 2. 이미지를 모델 입력 크기(384x384)로 변환하고, 텍스트를 토큰(숫자)으로 변환하는 전처리 과정을 포함합니다.
4	Code	CER(문자 오류율) 계산 함수: 모델이 예측한 글자와 정답 글자가 얼마나 다른지 측정하는 평가 지표 함수를 정의합니다.

5	Code	모델 학습 실행: 1. <code>ddobokki/ko-trocr</code> (한국어 사전학습 모델)을 불러옵니다. 2. 학습 파라미터(배치 크기, 학습률, 에폭 등)를 설정합니다(<code>Seq2SeqTrainingArguments</code>). 3. <code>Trainer</code> 를 통해 데이터셋으로 모델 학습을 진행하고 결과를 저장합니다.
6	Markdown	[단계 2] 성능 평가: 학습된 OCR 모델을 테스트합니다.
7	Code	평가용 클래스 정의 (<code>OCREvaluator</code>): 학습된 모델을 불러와서 이미지를 넣으면 텍스트를 뱉어내는 추론용 클래스를 만듭니다.
8	Code	개별 이미지 추론: 1. 검증 데이터 중 무작위로 몇 장을 뽑습니다. 2. 모델이 이미지를 보고 예측한 텍스트(Pred)와 실제 정답(Label)을 나란히 출력하여 비교합니다. 3. 개별 이미지의 CER 점수도 함께 보여줍니다.
9	Code	전체 성능 평가 (선택): 데이터셋 전체에 대해 정확도를 측정하는 코드로, 시간이 오래 걸려 주석 처리되어 있습니다. 필요시 주석을 풀고 실행하면 전체 데이터의 평균 오류율 등을 확인할 수 있습니다.